Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет прикладних систем та прикладної математики

ЗВІТ

з лабораторної роботи №1

з дисципліни: «Розробка програмного забезпечення систем реального часу»

Тема: «Вступ до ROS. Вузли.»

Виконав:
студент групи КВ-34
Турчик Тарас Іванович

Git-репозиторій: ros2-lab1

Київ — 2026

Мета роботи: Опанувати засоби контейнеризації середовища розробки (Docker), навчитися створювати та збирати ROS 2 пакети за допомогою утиліти colcon, проектувати вузли у вигляді компонентів (Components) для роботи у межах єдиного процесу-виконавця (Executor) та реалізовувати вузли з керованим життєвим циклом (Lifecycle Nodes).

Завдання 1.1: Налаштування середовища та робочого простору

Для ізоляції середовища розробки та забезпечення відтворюваності збірки створено Dockerfile на основі офіційного образу osrf/ros:humble-desktop. У контейнері встановлюються розширення python3-colcon-common-extensions та бібліотеки ros-humble-rclcpp-components, ros-humble-rclcpp-lifecycle і ros-humble-lifecycle-msgs, формується робоча тека /ros2_ws/src та налаштовується точка входу /entrypoint.sh. Скрипт entrypoint.sh автоматично виконує сорсинг системного /opt/ros/$ROS_DISTRO/setup.bash і локального /ros2_ws/install/setup.bash.

Dockerfile

```
FROM osrf/ros:humble-desktop

RUN apt-get update && apt-get install -y \
    python3-colcon-common-extensions \
    ros-humble-rclcpp-components \
    ros-humble-rclcpp-lifecycle \
    ros-humble-lifecycle-msgs \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /ros2_ws
RUN mkdir -p /ros2_ws/src

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
```

```
ENTRYPOINT ["/entrypoint.sh"]
CMD ["bash"]
```

entrypoint.sh

```
#!/bin/bash
set -e

source /opt/ros/$ROS_DISTRO/setup.bash

if [ -f /ros2_ws/install/setup.bash ]; then
   source /ros2_ws/install/setup.bash
fi

exec "$@"
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(lab1_pkg)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES
"Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_components REQUIRED)
find_package(rclcpp_lifecycle REQUIRED)
find_package(lifecycle_msgs REQUIRED)

# Завдання 1.2: Спільна бібліотека компонента
add_library(my_component SHARED src/my_component.cpp)
ament_target_dependencies(my_component rclcpp rclcpp_components)
```

```
rclcpp_components_register_nodes(my_component "lab1_ns::MyComponent")

# Завдання 1.3: Керовані вузли та супервізор
add_executable(managed_nodes_exec src/managed_node.cpp)
ament_target_dependencies(managed_nodes_exec rclcpp rclcpp_lifecycle lifecycle_msgs)

add_executable(lifecycle_supervisor src/lifecycle_supervisor.cpp)
ament_target_dependencies(lifecycle_supervisor rclcpp lifecycle_msgs)

install(TARGETS my_component
  ARCHIVE DESTINATION lib
  LIBRARY DESTINATION lib
  RUNTIME DESTINATION bin
)

install(TARGETS managed_nodes_exec lifecycle_supervisor
  DESTINATION lib/${PROJECT_NAME}
)

ament_package()
```

Завдання 1.2: Розробка вузла-компонента (Component Node)

У файлі src/my_component.cpp створено клас MyComponent у просторі імен lab1_ns, успадкований від rclcpp::Node. Його конструктор приймає const rclcpp::NodeOptions & options, ініціалізує вузол з ім'ям "my_component_node" і виводить у лог інформаційне повідомлення. Клас зареєстровано як компонент макросом RCLCPP_COMPONENTS_REGISTER_NODE(lab1_ns::MyComponent) та зібрано у спільну бібліотеку libmy_component.so.

my_component.cpp

```
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_components/register_node_macro.hpp"

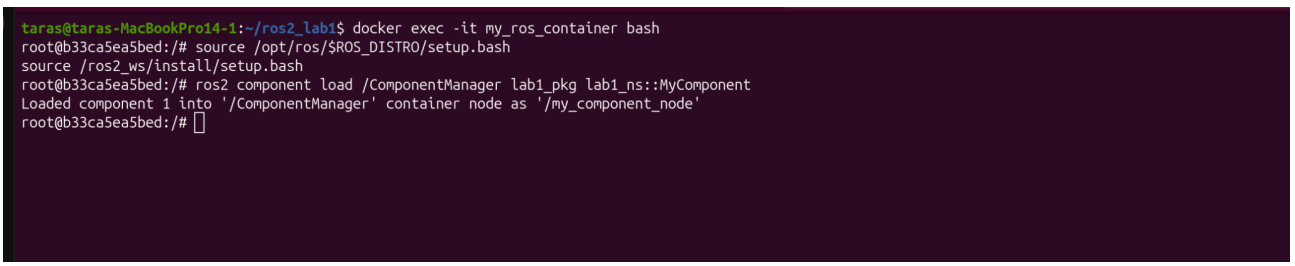
namespace lab1_ns
{

class MyComponent : public rclcpp::Node
{
public:
  explicit MyComponent(const rclcpp::NodeOptions & options)
  : Node("my_component_node", options)
  {
    RCLCPP_INFO(this->get_logger(), "Вузол-компонент MyComponent успішно ініціалізовано!");
  }
};

}

RCLCPP_COMPONENTS_REGISTER_NODE(lab1_ns::MyComponent)
```





Завдання 1.3: Розробка керованих вузлів (Lifecycle Nodes) та супервізора

У файлі src/managed_node.cpp створено два керовані вузли (managed_node_1 та managed_node_2) з використанням класу rclcpp_lifecycle::LifecycleNode. Налаштовано обробку переходів станів (on_configure, on_activate, on_deactivate, on_cleanup, on_shutdown). Вузли запущено у межах SingleThreadedExecutor, а їх послідовна конфігурація (перехід у стан Inactive) та активація (перехід у стан Active) виконані програмно за допомогою розробленого вузла-супервізора LifecycleSupervisor (src/lifecycle_supervisor.cpp) через сервіс /<node_name>/change_state.

managed_node.cpp

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_lifecycle/lifecycle_node.hpp"

using CallbackReturn =
rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;

class ManagedNode : public rclcpp_lifecycle::LifecycleNode
{
public:
  explicit ManagedNode(const std::string & node_name)
  : rclcpp_lifecycle::LifecycleNode(node_name)
  {
    RCLCPP_INFO(get_logger(), "[%s] створено (стан: Unconfigured).", node_name.c_str());
  }

  CallbackReturn on_configure(const rclcpp_lifecycle::State &) override
  {
    RCLCPP_INFO(get_logger(), "[%s] on_configure(): Конфігурація...", get_name());
    return CallbackReturn::SUCCESS;
  }
```

```
  CallbackReturn on_activate(const rclcpp_lifecycle::State & state) override
  {
    LifecycleNode::on_activate(state);
    RCLCPP_INFO(get_logger(), "[%s] on_activate(): Вузол АКТИВОВАНО!", get_name());
    return CallbackReturn::SUCCESS;
  }

  CallbackReturn on_deactivate(const rclcpp_lifecycle::State & state) override
  {
    LifecycleNode::on_deactivate(state);
    RCLCPP_INFO(get_logger(), "[%s] on_deactivate(): Деактивовано.", get_name());
    return CallbackReturn::SUCCESS;
  }

  CallbackReturn on_cleanup(const rclcpp_lifecycle::State &) override
  {
    RCLCPP_INFO(get_logger(), "[%s] on_cleanup(): Очищення ресурсів.", get_name());
    return CallbackReturn::SUCCESS;
  }

  CallbackReturn on_shutdown(const rclcpp_lifecycle::State &) override
  {
    RCLCPP_INFO(get_logger(), "[%s] on_shutdown(): Завершення роботи.", get_name());
    return CallbackReturn::SUCCESS;
  }
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  rclcpp::executors::SingleThreadedExecutor executor;
```

```
  auto node_1 = std::make_shared<ManagedNode>("managed_node_1");
  auto node_2 = std::make_shared<ManagedNode>("managed_node_2");

  executor.add_node(node_1->get_node_base_interface());
  executor.add_node(node_2->get_node_base_interface());

  executor.spin();
  rclcpp::shutdown();
  return 0;
}
```

lifecycle_supervisor.cpp

```
#include <chrono>
#include <memory>
#include <string>
#include <vector>

#include "rclcpp/rclcpp.hpp"
#include "lifecycle_msgs/msg/transition.hpp"
#include "lifecycle_msgs/srv/change_state.hpp"

using namespace std::chrono_literals;

class LifecycleSupervisor : public rclcpp::Node
{
public:
  LifecycleSupervisor() : Node("lifecycle_supervisor") {}

  bool change_state(const std::string & node_name, std::uint8_t transition_id)
  {
    std::string service_name = "/" + node_name + "/change_state";
    auto client = this->create_client<lifecycle_msgs::srv::ChangeState>(service_name);
```

```
    if (!client->wait_for_service(5s)) {
      RCLCPP_ERROR(get_logger(), "Сервіс %s недоступний!", service_name.c_str());
      return false;
    }

    auto request = std::make_shared<lifecycle_msgs::srv::ChangeState::Request>();
    request->transition.id = transition_id;

    auto future = client->async_send_request(request);
    if (rclcpp::spin_until_future_complete(this->get_node_base_interface(), future) ==
        rclcpp::FutureReturnCode::SUCCESS)
    {
      return future.get()->success;
    }
    return false;
  }
};

int main(int argc, char * argv[])
{
  rclcpp::init(argc, argv);
  auto supervisor = std::make_shared<LifecycleSupervisor>();

  std::vector<std::string> nodes = {"managed_node_1", "managed_node_2"};

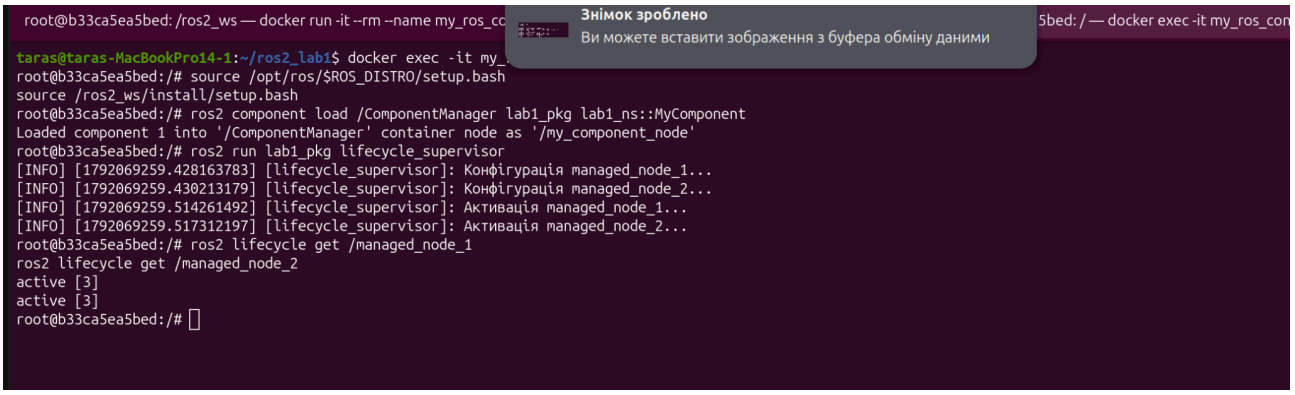
  for (const auto & name : nodes) {
    RCLCPP_INFO(supervisor->get_logger(), "Конфігурація %s...", name.c_str());
    supervisor->change_state(name,
lifecycle_msgs::msg::Transition::TRANSITION_CONFIGURE);
  }

  for (const auto & name : nodes) {
```

```
  RCLCPP_INFO(supervisor->get_logger(), "Активація %s...", name.c_str());
  supervisor->change_state(name, lifecycle_msgs::msg::Transition::TRANSITION_ACTIVATE);
 }

 rclcpp::shutdown();
 return 0;
}
```





Висновки: Під час виконання лабораторної роботи №1 опановано засоби контейнеризації середовища розробки ROS 2 за допомогою Docker та автоматичного скрипта entrypoint.sh. Створено і зібрано пакет lab1_pkg за допомогою утиліти colcon build --symlink-install. Реалізовано вузол-компонент lab1_ns::MyComponent у вигляді спільної бібліотеки libmy_component.so та перевірено його динамічне завантаження у процес-виконавець component_container. Також розроблено два керовані вузли (LifecycleNode) та вузол-супервізор, який успішно перевів їх зі стану Unconfigured у робочий стан Active (active [3]) за допомогою переходів TRANSITION_CONFIGURE і TRANSITION_ACTIVATE.

Контрольні запитання:

1. Які переваги та потенційні ризики виникають при запуску кількох вузлів-компонентів у межах одного процесу-виконавця (Executor)? *Переваги:* Суттєво знижуються затримки комунікації між вузлами за рахунок використання спільної пам'яті. *Ризики:* Критична помилка в одному з компонентів призведе до аварійного завершення всього контейнера та всіх суміжних вузлів у цьому процесі.

2. Які обов'язкові вимоги висуваються до C++ класу для можливості його експорту та динамічного завантаження як компонента ROS 2? Клас повинен бути успадкований від rclcpp::Node, його конструктор обов'язково має приймати аргумент rclcpp::NodeOptions. Також вузол необхідно зареєструвати макросом RCLCPP_COMPONENTS_REGISTER_NODE, а в CMakeLists.txt він має компілюватися у спільну бібліотеку (тип SHARED).

3. Чому виклик rclcpp::spin_once() у конструкторі Lifecycle-вузла є небезпечним і до яких критичних помилок це може призвести?

Це небезпечно, оскільки під час виконання конструктора об'єкт вузла ще не повністю створений і зовнішній std::shared_ptr ним ще не володіє, що призведе до винятку std::bad_weak_ptr. Крім того, це порушує модель скінченного автомата LifecycleNode і може спричинити взаємне блокування (deadlock) або аварійне завершення програми через спробу обробки колбеків до переходу вузла у початковий стан Unconfigured.

4. Опишіть основні стани та переходи скінченного автомата життєвого циклу керованого вузла (Lifecycle Node). *Основні стани:* Unconfigured (Неналаштований), Inactive (Неактивний), Active (Активний), Finalized (Завершений). *Основні переходи:* configure (Unconfigured -> Inactive), activate (Inactive -> Active), deactivate (Active -> Inactive), cleanup (Inactive -> Unconfigured) та shutdown (перехід із Unconfigured, Inactive або Active у Finalized).